data streams between services and Spring Cloud Function to define message handlers as functions. The latter approach could naturally evolve in serverless applications deployed to platforms like Azure Functions, AWS Lambda, or Knative.

DATA

Data is a crucial part of software systems. In Polar Bookshop you'll use the PostgreSQL relational database to permanently store the data processed by the application. I'll show you how to integrate an application with a data source using Spring Data JDBC (imperative) and Spring Data R2DBC (reactive). You'll then learn how to evolve a data source and manage schema migrations with Flyway.

Cloud native applications should be stateless, but the state needs to be stored somewhere. In Polar Bookshop you'll use Redis to externalize the session storage to a data store and keep applications stateless and scalable. Spring Session makes it easy to implement clustered user sessions. In particular, I'll show you how to use Spring Session Data Redis to integrate the application session management with Redis.

Besides persisted data and session data, you'll also deal with messages to implement event-driven programming patterns. Spring AMQP and RabbitMQ will be the technologies you'll use to do so.

Locally, you'll run these data services in Docker containers. In production, you'll rely on the managed services offered by a cloud provider like DigitalOcean or Azure, which take care of critical concerns like high availability, clustering, storage, and data replication.

CONFIGURATION